

2.1.1

1. 导入库

```
import pandas as pd
```

- 导入pandas库，用于数据处理和分析，通常别名为pd

2. 加载数据集

```
data = pd.read_csv('auto-mpg.csv')
print("数据集的前五行:")
print(data.head())
```

- `pd.read_csv()`: 读取CSV文件
 - 参数: 文件路径('auto-mpg.csv')
- `data.head()`: 显示数据框的前5行
- 功能: 加载数据集并预览前5行数据

3. 检查数据类型

```
print(data.dtypes)
```

- `data.dtypes`: 显示每一列的数据类型
- 功能: 了解各列的数据类型

4. 检查和处理缺失值

```
print("\n检查缺失值:")
print(data.isnull().sum())
data = data.dropna()
```

- `data.isnull().sum()`: 统计每列的缺失值数量
- `data.dropna()`: 删除包含缺失值的行
 - 默认删除任何包含NaN的行
- 功能: 检查并删除缺失值

5. 转换horsepower列

```
data['horsepower'] = pd.to_numeric(data['horsepower'], errors='coerce')
data = data.dropna(subset=['horsepower'])
```

- `pd.to_numeric()`: 将列转换为数值类型
 - `errors='coerce'`: 将无法转换的值设为NaN

- `dropna(subset=['horsepower'])`: 只删除horsepower列中的缺失值
- 功能: 处理horsepower列中的非数值数据

6. 再次检查数据类型

```
print(data.horsepower.dtypes)
```

- 检查horsepower列的数据类型是否已转换为数值型

7. 检查清洗后的缺失值

```
print("\n检查清洗后的缺失值:")  
print(data.isnull().sum())
```

- 确认数据清洗后是否还有缺失值

8. 数据标准化

```
from sklearn.preprocessing import StandardScaler  
numerical_features = ['displacement', 'horsepower', 'weight', 'acceleration']  
scaler = StandardScaler()  
data[numerical_features] = scaler.fit_transform(data[numerical_features])
```

- `StandardScaler()`: 创建标准化对象
 - 标准化公式: $(x - \text{mean}) / \text{std}$
- `fit_transform()`: 计算均值和标准差并应用转换
- 功能: 对数值特征进行标准化处理

9. 划分特征和目标变量

```
from sklearn.model_selection import train_test_split  
selected_features = ['cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model  
year', 'origin']  
X = data[selected_features]  
y = data['mpg']
```

- `X`: 特征矩阵(自变量)
- `y`: 目标变量(mpg, 因变量)
- 功能: 选择建模使用的特征和目标

10. 划分训练集和测试集

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- `train_test_split()`: 划分数据集
 - `test_size=0.2`: 测试集占20%

- `random_state=42`: 随机种子(保证可重复性)
- 功能: 将数据划分为训练集和测试集

11. 合并特征和目标变量

```
cleaned_data = X.copy()
cleaned_data['mpg'] = y
```

- 将处理后的特征和目标合并到一个DataFrame中

12. 保存清洗后的数据

```
cleaned_data.to_csv('2.1.1_cleaned_data.csv', index=False)
print("\n清洗后的数据已保存到 2.1.1_cleaned_data.csv")
```

- `to_csv()`: 保存DataFrame为CSV文件
 - `index=False`: 不保存行索引
- 功能: 保存处理后的数据

主要数据处理流程:

1. 数据加载和初步检查
2. 缺失值处理
3. 数据类型转换
4. 数据标准化
5. 特征选择
6. 数据集划分
7. 数据保存

2.1.2

下面我将详细解释每一行代码的功能、函数的作用以及参数的含义:

1. 导入库

```
import pandas as pd
```

- 导入pandas库, 用于数据处理和分析, 通常别名为pd

2. 加载Excel数据集

```
data = pd.read_excel('大学生低碳生活行为的影响因素数据集.xlsx')
print(data.head())
```

- `pd.read_excel()`: 读取Excel文件
 - 参数: 文件路径('大学生低碳生活行为的影响因素数据集.xlsx')
- `data.head()`: 显示数据框的前5行
- 功能: 加载数据集并预览前5行数据

3. 缺失值处理

```
initial_row_count = data.shape[0] # 获取原始数据行数
data = data.dropna() # 删除包含缺失值的行
final_row_count = data.shape[0] # 获取处理后数据行数
print(f'处理后数据行数: {final_row_count}, 删除的行数: {initial_row_count - final_row_count}')
```

- `data.shape[0]`: 获取DataFrame的行数
- `data.dropna()`: 删除包含任何缺失值的行
 - 默认删除任何包含NaN的行
- 功能: 统计并删除缺失值, 打印处理前后的数据量变化

4. 删除重复行

```
data = data.drop_duplicates()
```

- `data.drop_duplicates()`: 删除完全重复的行
- 功能: 确保数据集中没有重复记录

5. 数据标准化

```
from sklearn.preprocessing import StandardScaler
numerical_features = ['4. 您的月生活费○≤1,000元    ○1,001-2,000元    ○2,001-3,000元    ○≥3,001元']
scaler = StandardScaler()
data[numerical_features] = scaler.fit_transform(data[numerical_features])
```

- `StandardScaler()`: 创建标准化对象
 - 标准化公式: $(x - \text{mean}) / \text{std}$
- `fit_transform()`: 计算均值和标准差并应用转换
- 功能: 对数值特征(月生活费)进行标准化处理

6. 特征选择

```
selected_features = ['1.您的性别○男性    ○女性',
                    '2.您的年级○大一    ○大二    ○大三    ○大四',
                    '3.您的源地○农村    ○城镇（乡镇）    ○地县级城市    ○省会城市及直辖市',
                    '4.您的月生活费○≤1,000元    ○1,001-2,000元    ○2,001-3,000元    ○≥3,001元',
                    '5.您进行过绿色低碳的相关生活方式吗?',
                    '6.您觉得"低碳", 与你的生活关系密切吗?',
                    '7.低碳生活是否会成为未来的主流生活方式?',
                    '8.您是否认为低碳生活会提高您的生活质量?']

x = data[selected_features]
```

- `x`: 特征矩阵(自变量)
- 功能: 选择用于建模的特征列

7. 创建目标变量

```
y = data['低碳行为积极性']
```

- `y`: 目标变量(因变量)
- 功能: 指定预测目标

8. 划分训练集和测试集

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- `train_test_split()`: 划分数据集
 - `x`: 特征数据
 - `y`: 目标数据
 - `test_size=0.2`: 测试集占20%
 - `random_state=42`: 随机种子(保证可重复性)
- 功能: 将数据划分为训练集(80%)和测试集(20%)

9. 合并并保存处理后的数据

```
cleaned_data = pd.concat([X, y], axis=1)
cleaned_data.to_csv('2.1.2_cleaned_data.csv', index=False)
```

- `pd.concat()`: 沿列方向(axis=1)合并特征和目标
- `to_csv()`: 保存为CSV文件
 - `index=False`: 不保存行索引
- 功能: 保存处理后的完整数据集

代码执行流程总结：

1. 数据加载：读取Excel文件
2. 数据清洗：
 - 处理缺失值
 - 删除重复值
3. 特征工程：
 - 数值特征标准化
 - 特征选择
4. 数据准备：
 - 定义目标变量
 - 划分训练/测试集
5. 数据保存：
 - 合并处理后的数据
 - 保存为CSV文件

2.1.3

1. 导入库

```
import pandas as pd
```

- 导入pandas库，用于数据处理和分析，通常别名为pd

2. 加载数据

```
data = pd.read_csv('finance数据集.csv')  
data.head()
```

- `pd.read_csv()`：读取CSV文件
 - 参数：文件路径('finance数据集.csv')
- `data.head()`：显示数据框的前5行
- 功能：加载数据集并预览数据

3. 导入可视化库

```
import matplotlib.pyplot as plt  
import seaborn as sns
```

- `matplotlib.pyplot`：Python绘图库

- `seaborn`: 基于matplotlib的统计数据可视化库

4. 绘制箱线图

```
plt.figure(figsize=(12, 8))
numeric_cols = data.select_dtypes(include=['float64', 'int64']).columns
for i, col in enumerate(numeric_cols, 1):
    plt.subplot(3, 4, i)
    sns.boxplot(x=data[col])
    plt.title(col)
plt.tight_layout()
plt.show()
```

- `plt.figure(figsize=(12, 8))`: 设置图形大小(宽12英寸, 高8英寸)
- `select_dtypes()`: 选择指定数据类型的列
 - `include=['float64', 'int64']`: 选择数值型列
- `enumerate(numeric_cols, 1)`: 遍历数值列, 从1开始计数
- `plt.subplot(3, 4, i)`: 创建3行4列的子图网格
- `sns.boxplot()`: 绘制箱线图
 - `x=data[col]`: 指定绘图数据
- `plt.title(col)`: 设置子图标题为列名
- `plt.tight_layout()`: 自动调整子图参数
- `plt.show()`: 显示图形
- 功能: 可视化数值特征的分布和异常值

5. IQR处理异常值

```
Q1 = data[numeric_cols].quantile(0.25)
Q3 = data[numeric_cols].quantile(0.75)
IQR = Q3 - Q1
data_cleaned = data[~((data[numeric_cols] < (Q1 - 1.5 * IQR)) | (data[numeric_cols] > (Q3 + 1.5 * IQR)))].any(axis=1)]
```

- `quantile(0.25) / quantile(0.75)`: 计算25%和75%分位数
- $IQR = Q3 - Q1$: 计算四分位距
- 异常值判定条件: 小于 $Q1 - 1.5IQR$ 或大于 $Q3 + 1.5IQR$
- `any(axis=1)`: 检查每行是否有任何列满足条件
- `~`: 逻辑取反, 保留非异常值
- 功能: 基于IQR方法识别并删除异常值

6. 处理重复值

```
duplicates = data_cleaned.duplicated()
num_duplicates = duplicates.sum()
data_cleaned = data_cleaned[~duplicates]
print(f'删除的重复行数: {num_duplicates}')
```

- `data.duplicated()`: 标识重复行
- `sum()`: 统计重复行数
- `~duplicates`: 保留非重复行
- 功能: 删除完全重复的行并打印删除数量

7. 数据归一化

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
data_cleaned[numeric_cols] = scaler.fit_transform(data_cleaned[numeric_cols])
```

- `MinMaxScaler()`: 创建最小-最大缩放器
 - 归一化公式: $(x - \min) / (\max - \min)$
- `fit_transform()`: 计算最小/最大值并应用转换
- 功能: 将数值特征缩放到[0,1]范围

8. 设定目标变量

```
target_variable = 'SeriousDlqin2yrs'
```

- 指定目标变量列名

9. 划分特征和目标

```
X = data_cleaned.drop(columns=[target_variable])
y = data_cleaned[target_variable]
```

- `drop()`: 删除指定列(目标变量)得到特征矩阵
- `y`: 提取目标变量
- 功能: 分离特征(X)和目标(y)

10. 划分训练/测试集

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f'训练数据形状: {X_train.shape}')
print(f'测试数据形状: {X_test.shape}')
```

- `train_test_split()`: 划分数据集

- `test_size=0.2`: 测试集占20%
- `random_state=42`: 随机种子
- `shape`: 打印数据维度
- 功能: 按80/20比例划分训练/测试集

11. 保存清洗后的数据

```
cleaned_file_path = '2.1.3_cleaned_data.csv'  
data_cleaned.to_csv(cleaned_file_path, index=False)
```

- `to_csv()`: 保存为CSV文件
 - `index=False`: 不保存行索引
- 功能: 保存处理后的数据集

完整处理流程:

1. 数据加载与探索
2. 异常值检测与处理(IQR方法)
3. 重复值处理
4. 数据归一化(MinMaxScaler)
5. 特征与目标分离
6. 训练/测试集划分
7. 数据保存

这个代码实现了一个金融数据集的完整预处理流程, 特别关注了异常值处理和特征缩放, 为后续的信用风险建模做好准备。关键点包括:

- 使用箱线图直观展示数据分布
- 基于统计方法(IQR)的异常值处理
- 最小-最大归一化保持数据尺度一致
- 规范的训练/测试集划分

2.1.4

下面我将详细解释每一行代码的功能、函数的作用以及参数的含义:

1. 导入库

```
import pandas as pd
```

- 导入pandas库, 用于数据处理和分析, 通常别名为pd

2. 加载数据集

```
data = pd.read_csv('medical_data.csv', encoding='gbk')
```

- `pd.read_csv()`: 读取CSV文件
 - `'medical_data.csv'`: 文件路径
 - `encoding='gbk'`: 指定文件编码格式(处理中文)

3. 数据探索

```
print(data.dtypes)
print(data.head())
```

- `dtypes`: 显示每列的数据类型
- `head()`: 显示前5行数据
- 功能: 初步了解数据结构和内容

4. 检查缺失值

```
print(data.isnull().sum())
```

- `isnull().sum()`: 统计每列的缺失值数量
- 功能: 识别数据完整性问题

5. 日期格式处理

```
data['就诊日期'] = pd.to_datetime(data['就诊日期'])
data['诊断日期'] = pd.to_datetime(data['诊断日期'])
```

- `pd.to_datetime()`: 将列转换为datetime类型
- 功能: 标准化日期格式以便后续计算

6. 列名修改

```
data.rename(columns={'病人ID': '患者ID'}, inplace=True)
```

- `rename()`: 修改列名
 - `columns={'病人ID': '患者ID'}`: 指定旧名和新名的映射
 - `inplace=True`: 直接修改原DataFrame
- 功能: 统一命名规范

7. 新增计算列

```
from datetime import datetime
data['诊断延迟'] = (data['诊断日期'] - data['就诊日期']).dt.days
data['病程'] = (datetime(2024, 9, 1) - data['诊断日期']).dt.days
```

- `dt.days`: 提取时间差的天数部分
- `datetime(2024, 9, 1)`: 设定参考日期
- 功能: 计算诊断延迟天数和病程天数

8. 数据清洗

```
data = data[(data['诊断延迟'] >= 0) & (data['年龄'] > 0) & (data['年龄'] < 120)]
```

- 布尔索引过滤:
 - 诊断延迟不小于0天
 - 年龄在0-120岁之间
- 功能: 删除不合理数据

9. 数据统计

```
print(data.describe())
```

- `describe()`: 生成数值列的统计摘要
- 功能: 查看清洗后数据的统计特征

10. 处理重复值

```
initial_rows = data.shape[0]
data.drop_duplicates(inplace=True)
deleted_rows = initial_rows - data.shape[0]
print(f'删除的重复行数: {deleted_rows}')
```

- `shape[0]`: 获取行数
- `drop_duplicates()`: 删除完全重复的行
 - `inplace=True`: 直接修改原DataFrame
- 功能: 删除重复记录并统计数量

11. 数据归一化

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
columns_to_normalize = ['年龄', '体重', '身高']
data[columns_to_normalize] = scaler.fit_transform(data[columns_to_normalize])
```

- `MinMaxScaler()`: 创建最小-最大缩放器
 - 归一化公式: $(x - \min) / (\max - \min)$
- `fit_transform()`: 计算并应用转换
- 功能: 将数值特征缩放到[0,1]范围

12. 数据可视化准备

```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
font_path = 'C:/Windows/Fonts/simhei.ttf'
my_font = fm.FontProperties(fname=font_path)
```

- 设置中文字体显示
 - `simhei.ttf`: 黑体字体文件路径

13. 治疗结果分布分析

```
treatment_outcome_distribution = data.groupby('疾病类型')['治疗结果'].value_counts().unstack()
treatment_outcome_distribution.plot(kind='bar', stacked=True)
```

- `groupby().value_counts().unstack()`: 创建交叉频数表
- `plot(kind='bar', stacked=True)`: 绘制堆叠柱状图
- 功能: 展示不同疾病类型的治疗结果分布

14. 年龄与疾病严重程度分析

```
plt.scatter(data['年龄'], data['疾病严重程度'])
```

- `scatter()`: 绘制散点图
- 功能: 探索年龄与疾病严重程度的关系

15. 图表美化

```
plt.title()/plt.xlabel()/plt.ylabel(): 设置标题和轴标签
plt.xticks()/plt.yticks(): 设置刻度标签
plt.legend(): 添加图例
```

- `fontproperties=my_font`: 指定中文字体
- 功能: 确保图表正确显示中文

16. 保存处理后的数据

```
output_path = '2.1.4_cleaned_data.csv'
data.to_csv(output_path, index=False)
```

- `to_csv()`: 保存为CSV文件
 - `index=False`: 不保存行索引
- 功能: 导出清洗后的数据

完整处理流程:

1. 数据加载与编码处理
2. 数据质量检查(缺失值、异常值)
3. 日期格式标准化
4. 特征工程(新增计算列)
5. 数据清洗(删除不合理值和重复值)
6. 特征归一化
7. 数据可视化分析
8. 结果保存

2.1.5

下面我将详细解释每一行代码的功能、函数的作用以及参数的含义:

1. 导入库

```
import pandas as pd
```

- 导入pandas库, 用于数据处理和分析, 别名为pd

2. 加载数据集

```
data = pd.read_csv('健康咨询客户数据集.csv')
```

- `pd.read_csv()`: 读取CSV文件
 - 参数: 文件路径('健康咨询客户数据集.csv')

3. 查看数据基本信息

```
print(data.info())
```

- `info()`: 显示数据集的简明摘要
 - 包括列名、非空值数量、数据类型等

4. 检查缺失值

```
print(data.isnull().sum())
```

- `isnull().sum()`: 统计每列的缺失值数量

5. 删除缺失值

```
data_cleaned = data.dropna()
```

- `dropna()`: 删除包含任何缺失值的行
 - 默认删除任何包含NaN的行

6. 处理年龄列

```
data_cleaned.loc[:, 'Your age'] = pd.to_numeric(data_cleaned['Your age'], errors='coerce')
data_cleaned = data_cleaned.dropna(subset=['Your age'])
data_cleaned = data_cleaned[data_cleaned['Your age'] >= 0]
data_cleaned.loc[:, 'Your age'] = data_cleaned['Your age'].astype(int)
```

- `pd.to_numeric()`: 将列转换为数值类型
 - `errors='coerce'`: 将无法转换的值设为NaN
- `dropna(subset=['Your age'])`: 只删除age列中的缺失值
- `data_cleaned['Your age'] >= 0`: 过滤掉负年龄
- `astype(int)`: 转换为整数类型

7. 检查数据类型

```
print(data_cleaned['Your age'].dtype)
```

- 打印age列的数据类型

8. 处理重复值

```
duplicates_removed = data_cleaned.duplicated().sum()
data_cleaned = data_cleaned.drop_duplicates()
print(f"Removed {duplicates_removed} duplicate rows")
```

- `duplicated().sum()`: 统计重复行数
- `drop_duplicates()`: 删除重复行
- 打印删除的重复行数

9. 标签编码

```
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
data_cleaned['How do you describe your current level of fitness ?'] =
label_encoder.fit_transform(data_cleaned['How do you describe your current level of fitness
?'])
```

- `LabelEncoder()`: 创建标签编码器
- `fit_transform()`: 将分类值转换为数字标签

10. 查看编码结果

```
print(data_cleaned['How do you describe your current level of fitness ?'].unique())
```

- `unique()`: 打印列的唯一值

11. 列名处理

```
data.columns = data.columns.str.strip()
print(data.columns)
```

- `str.strip()`: 去除列名两端的空格
- 打印处理后的列名

12. 特定列缺失值处理

```
data_cleaned = data.dropna(subset=['How often do you exercise?'])
```

- 只删除exercise列中的缺失值

13. 统计健身频率

```
exercise_frequency_counts = data_cleaned['How often do you exercise?'].value_counts()
```

- `value_counts()`: 统计每个值的出现次数

14. 绘制饼图

```
plt.figure(figsize=(10, 6))
exercise_frequency_counts.plot.pie(autopct='%1.1f%%', startangle=90,
colors=plt.cm.Paired.colors)
plt.title('Distribution of Exercise Frequency')
plt.ylabel('')
plt.show()
```

- `figure(figsize=(10, 6))`: 设置图形大小
- `plot.pie()`: 绘制饼图
 - `autopct='%1.1f%%'`: 显示百分比
 - `startangle=90`: 起始角度
 - `colors=plt.cm.Paired.colors`: 颜色方案
- `title()`: 设置标题
- `show()`: 显示图形

15. 填充缺失值

```
data_filled = data.apply(lambda x: x.fillna(x.mode()[0]))
```

- `apply()`: 对每列应用函数
- `fillna(x.mode()[0])`: 用众数填充缺失值

16. 划分数据集

```
train_data, test_data = train_test_split(data_filled, test_size=0.2, random_state=42)
```

- `train_test_split()`: 划分训练/测试集
 - `test_size=0.2`: 测试集占比20%
 - `random_state=42`: 随机种子

17. 保存数据

```
cleaned_file_path = '2.1.5_cleaned_data.csv'
data_filled.to_csv(cleaned_file_path, index=False)
```

- `to_csv()`: 保存为CSV文件
 - `index=False`: 不保存行索引

完整处理流程：

1. 数据加载与初步检查
2. 缺失值处理
3. 数据类型转换与异常值处理
4. 重复值处理
5. 分类数据编码
6. 数据可视化分析
7. 数据集划分
8. 结果保存

这个代码实现了一个健康咨询客户数据集的完整预处理流程，特别关注了：

- 数据质量检查与清洗
- 分类数据的标签编码
- 健身频率的可视化分析
- 完整的数据预处理流水线